

Collaborator Onboarding Guide

How to set up GitHub CLI and Claude Code to contribute to the

Training Package for Replication

Organization: easvrn • Repository: Training-Package-for-Replication

Admin contact: john.punzalan@eisnetwork.co

Table of Contents

Section 1 — Prerequisites

Before you begin, make sure you have the following:

- A computer running macOS or Windows
- An active internet connection
- A GitHub account (see below if you need to create one)
- An invitation to the easvrn organization repository from the admin

1.1 — Create a GitHub Account (if you don't have one)

If you already have a GitHub account, skip to Section 2.

1. Open your web browser and go to <https://github.com>
2. Click the green "Sign up" button in the top right
3. Enter your email address, create a password, and choose a username
4. Complete the verification puzzle and click Create account
5. Open the confirmation email from GitHub and click the verification link
6. You are now ready to use GitHub

1.2 — Accept Your Repository Invitation

The admin will send you an email invitation to join the easvrn organization.

1. Open the invitation email from GitHub
2. Click "Join easvrn" or "Accept invitation"
3. You now have access to the Training Package for Replication repository

Section 2 — Install Homebrew (macOS Only)

Note: Windows users: skip this section entirely and go directly to Section 3b.

Homebrew is a package manager for macOS that makes it easy to install developer tools like GitHub CLI and Node.js.

2.1 — Installation Steps

1. Open Terminal: press Cmd + Space, type "Terminal", then press Enter
2. Paste the following command and press Enter:

```
/bin/bash -c "$(curl -fsSL  
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```
3. Follow the on-screen instructions. You may be asked to enter your Mac login password — this is normal
4. Wait for the installation to complete (this may take a few minutes)
5. Verify the installation by running:

```
brew --version
```

You should see output like: Homebrew 4.x.x

Section 3 — Install GitHub CLI

GitHub CLI (gh) allows you to interact with GitHub directly from your terminal or command prompt — no browser needed for most tasks.

3a — macOS Installation

1. In Terminal, run:

```
brew install gh
```

2. Wait for the installation to complete

3. Verify the installation:

```
gh --version
```

You should see something like: gh version 2.x.x

3b — Windows Installation

1. Open your browser and go to <https://cli.github.com>
2. Click "Download for Windows"
3. Run the downloaded .msi installer file and follow the setup wizard
4. Open Command Prompt: press Win + R, type "cmd", and press Enter
5. Verify the installation:

```
gh --version
```

You should see something like: gh version 2.x.x

Section 4 — Authenticate GitHub CLI

Now that GitHub CLI is installed, you need to link it to your GitHub account.

4.1 — Authentication Steps

1. In Terminal (macOS) or Command Prompt (Windows), run:

```
gh auth login
```
2. When asked "Where do you use GitHub?", choose: GitHub.com
3. When asked "What is your preferred protocol?", choose: HTTPS
4. When asked "Authenticate Git with your GitHub credentials?", answer: Yes
5. When asked "How would you like to authenticate?", choose: Login with a web browser
6. A one-time code will appear in your terminal — copy it
7. Press Enter — your browser will open automatically
8. Paste the code on the GitHub authorization page and click Authorize
9. Return to your terminal — you should see: Logged in as [your username]

4.2 — Verify Authentication

Run the following to confirm everything is working:

```
gh auth status
```

You should see your username and a confirmation that you are logged in.

Note: If your browser does not open automatically, manually go to <https://github.com/login/device> and enter the one-time code from your terminal.

Section 5 — Install Claude Code

Claude Code is an AI coding assistant that runs directly in your terminal. It helps you edit files, write content, understand code, and much more — all by chatting in plain language.

5.1 — Install Node.js

Claude Code requires Node.js. Check if you already have it:

```
node --version
```

If you see a version number (e.g. v18.x.x), skip to Section 5.2.

If Node.js is not installed:

- macOS: run in Terminal:

```
brew install node
```
- Windows: go to <https://nodejs.org>, download the LTS version, and run the installer

5.2 — Install Claude Code

1. In Terminal or Command Prompt, run:

```
npm install -g @anthropic-ai/claude-code
```

2. Wait for installation to complete
3. Verify the installation:

```
claude --version
```

5.3 — First Launch and Login

1. Run Claude Code for the first time:

```
claude
```
2. Follow the on-screen prompts to log in with your Anthropic account
3. Once logged in, Claude is ready to use

Note: If you don't have an Anthropic account, you will be prompted to create one at claude.ai.

Section 6 — Clone the Repository

Cloning means downloading a copy of the project to your computer so you can work on it locally.

6.1 — Clone the Project

1. Open Terminal (macOS) or Command Prompt (Windows)
2. Navigate to the folder where you want to save the project:

```
cd ~/Documents
```

3. Clone the repository:

```
git clone https://github.com/easvrm/Training-Package-for-Replication.git
```

4. Enter the project folder:

```
cd Training-Package-for-Replication
```

5. Verify the contents were downloaded:

```
ls
```

You should see the project files listed in your terminal.

Note: *You only need to clone the repository once. After that, you use `git pull` to get updates (see Section 7).*

Section 7 — Daily Workflow

This is the process you will follow every time you want to make changes to the project. Follow these steps in order every time.

Step 1 — Open the Project in Claude Code

Navigate to the project folder and launch Claude:

```
cd ~/Documents/Training-Package-for-Replication
claude
```

You are now in Claude Code and can start asking Claude for help.

Step 2 — Always Pull the Latest Version First

Before making any changes, always download the latest version to avoid conflicts:

```
git pull origin master
```

Note: *Never skip this step. If someone else made changes since you last worked, this ensures you have their updates.*

Step 3 — Create Your Working Branch

A branch is your own personal workspace. It lets you make changes without affecting the main project until your work is reviewed.

```
git checkout -b feature/your-name-topic
```

Replace your-name-topic with something descriptive. Examples:

```
git checkout -b feature/alice-module2-update
git checkout -b feature/bob-fix-broken-link
```

Step 4 — Make Your Changes Using Claude

Inside Claude Code, you can ask Claude for help in plain English. For example:

- "Update the introduction text in songa-docs.html to mention the new module"
- "Fix the broken image reference on page 5"
- "Add a new section about field data collection"

Claude will read the files, suggest changes, and apply them for you.

Step 5 — Stage and Commit Your Changes

When you are happy with your changes, save them to Git:

```
git add .  
git commit -m "Brief description of your change"
```

Examples of good commit messages:

```
"Add facilitator notes to Module 3"  
"Fix broken image on page 12"  
"Update contact information in footer"
```

Step 6 — Push Your Branch to GitHub

Upload your branch to GitHub so it can be reviewed:

```
git push origin feature/your-name-topic
```

Replace feature/your-name-topic with the actual name of your branch.

Step 7 — Open a Pull Request

A Pull Request (PR) is how you ask the admin to review and approve your changes:

```
gh pr create --title "Your title" --body "What you changed and why"
```

Example:

```
gh pr create --title "Add Module 3 facilitator notes" --body "Added  
detailed notes for the facilitator section of Module 3 as requested."
```

Step 8 — Wait for Review

The admin will receive a notification and review your Pull Request. They may:

- Approve and merge your changes into the main project
- Leave comments asking for adjustments
- Request changes before merging

You will receive an email notification when your PR is reviewed.

Section 8 — Quick Command Reference

Use this table as a cheat sheet when working on the project.

Command	What it does
<code>git status</code>	Shows what files have changed
<code>git pull origin master</code>	Downloads the latest version from GitHub
<code>git checkout -b feature/name</code>	Creates a new working branch
<code>git add .</code>	Stages all modified files for commit
<code>git commit -m "message"</code>	Saves your changes with a description
<code>git push origin branch-name</code>	Uploads your branch to GitHub
<code>gh pr create</code>	Opens a Pull Request for review
<code>gh auth status</code>	Checks if you are logged in to GitHub CLI
<code>claude</code>	Opens Claude Code in the current folder

Section 9 — Troubleshooting

If something goes wrong, check the table below for common issues and solutions.

"command not found: gh"

GitHub CLI is not installed or not recognized.

- macOS: go back to Section 3a and run `brew install gh`
- Windows: go back to Section 3b and reinstall from <https://cli.github.com>

"command not found: claude"

Claude Code is not installed.

- Go back to Section 5 and run: `npm install -g @anthropic-ai/claude-code`

"Permission denied" or "Repository not found"

You may not have been added to the repository yet.

- Check your email for a GitHub invitation from the admin
- Contact the admin at john.punzalan@eisnetwork.co

Merge Conflict

A merge conflict happens when two people edited the same part of a file. Ask Claude for help:

Inside your project folder, open Claude and say:

```
"I have a merge conflict in [filename]. Help me fix it."
```

Claude will guide you through resolving it step by step.

Browser Does Not Open During gh auth login

If your browser does not open automatically during authentication:

- Manually open your browser
- Go to: <https://github.com/login/device>
- Enter the one-time code displayed in your terminal

"node: command not found"

Node.js is not installed.

- macOS: run brew install node
- Windows: download from <https://nodejs.org> (LTS version)

Section 10 — Getting Help

Ask Claude Directly

The fastest way to get help is to ask Claude inside your project folder:

```
cd ~/Documents/Training-Package-for-Replication
claude
```

Then type your question in plain English. Claude can help with any task related to the project.

Report an Issue on GitHub

If you find a bug, broken link, or content error, open an issue on the repository:

Repository Issues: <https://github.com/easvrn/Training-Package-for-Replication/issues>

Contact the Project Admin

Email: john.punzalan@eisnetwork.co

Organization: easvrn on GitHub

You're all set! Welcome to the team.